Functions and Function Pointers Assessment

Name: _____

Registration Number: _____

Institute: ☐ KCE ☐ KIT ☐ KAHE

1. `int fun(int x){ return x*x; } printf("%d", fun(5));` **Output?**

- ○ 5
- ○ 10
- ○ 25
- ○ Compilation Error

2. Which of the following is the correct function prototype?

- ○ int sum(int,int);
- ○ sum(int,int);
- ○ function sum(int,int);
- ○ int sum();

3. A function that does not return any value should have the return type

- ○ NULL
- ○ int
- ○ void
- ○ empty

4. `void fun(){ return 10; }` **The above code**

- Works correctly
- Compilation Error
- Returns 10
- Warning only

5. Which keyword indicates that a function accepts no arguments?

- void fun(void);
- void fun();
- fun(void);
- void();

6. What is the default return type of a function in modern C (C99 and later) if omitted?

- int
- void
- float
- Compilation Error

7. Functions in C are generally passed arguments

- By reference
- By value
- By address automatically
- By copy constructor

8. Which memory stores local variables?

- Heap
- ROM
- Stack
- Data Segment

9. Each function call creates a new

- ○ Heap Block
- ○ Stack Frame
- ○ Structure
- ○ Register

10. Which information is stored in a stack frame?

- ○ Local variables
- ○ Return address
- ○ Parameters
- ○ All of the above

11. What happens if recursive calls never reach a base case?

- ○ Infinite loop
- ○ Stack Overflow
- ○ Memory Leak
- ○ Segmentation Fault during compilation

12. `void fun(){ fun(); }` **The program results in**

- ○ Infinite recursion
- ○ Compile Error
- ○ Syntax Error
- ○ Returns normally

13. The stopping condition in recursion is called

- ○ Exit Point
- ○ Base Case
- ○ End Loop
- ○ Return Block

14. Recursion primarily uses

- Queue
- Heap
- Stack
- Array

15. `int f(int n){ if(n==0) return 1; return n*f(n-1); }` **What does f(4) return?**

- 10
- 24
- 16
- 120

16. Function overloading in C is

- Supported
- Not supported
- Compiler dependent
- Optional

17. Which is a valid function pointer declaration?

- int (*fp)(int);
- int *fp(int);
- int fp*(int);
- pointer fp(int);

18. `int add(int a,int b){ return a+b; } int (*fp)(int,int)=add; printf("%d",fp(5,6));` **Output?**

- 30
- 11
- Error
- 56

19. Which statement also invokes the function pointer?

- (*fp)(5,6);
- fp->(5,6);
- *fp(5,6);
- fp->call();

20. A function pointer stores

- Variable
- Address of function
- Structure
- String

21. Function pointers are commonly used in

- Callback functions
- Sorting libraries
- Interrupt handlers
- All of the above

22. `void (*fp)(void);` **means**

- Function returning pointer
- Pointer to function
- Pointer returning function
- Invalid

23. Can a function return a pointer?

- Yes
- No
- Only C++
- Only Linux

24. Can a function return another function?

- Yes
- No
- Only using pointers
- Only recursion

25. Which function can legally return NULL?

- int
- Pointer
- float
- char

26. The return address is stored in

- Heap
- Stack Frame
- ROM
- Global Memory

27. The lifetime of local variables ends when

- ○ Program ends
- ○ Function returns
- ○ main() ends
- ○ Compiler exits

28. Global variables are stored in

- ○ Stack
- ○ Heap
- ○ Data Segment
- ○ Registers

29. `void fun(int a[]){` **Inside the function, a is actually**

- ○ Array
- ○ Pointer
- ○ Structure
- ○ Integer

30. Which recursion is generally more memory efficient?

- ○ Tail Recursion
- ○ Mutual Recursion
- ○ Nested Recursion
- ○ Binary Recursion

31. Mutual recursion means

- ○ Function calling itself
- ○ Two or more functions calling each other
- ○ Recursive loop
- ○ Function pointer recursion

32. A callback function is implemented using

- Structure
- Function Pointer
- Union
- Array

33. `int (*fp[3])(int);` **declares**

- Array of function pointers
- Pointer to array
- Array of functions
- Function returning array

34. `int (*(*fp)(void))(int);` **This declaration represents**

- Function pointer
- Function returning function pointer
- Pointer to integer
- Array pointer

35. Which standard library function heavily uses function pointers?

- printf()
- scanf()
- qsort()
- strcpy()

36. The comparison function passed to qsort() is

- Integer
- Callback Function
- Macro
- Structure

37. What happens if a recursive function allocates a large local array?

- ○ Faster execution
- ○ Increased stack usage
- ○ Heap expansion
- ○ No effect

38. Which memory error is common in deep recursion?

- ○ Heap Overflow
- ○ Stack Overflow
- ○ Double Free
- ○ Memory Leak

39. A static local variable is stored in

- ○ Stack
- ○ Heap
- ○ Data Segment
- ○ Register

40. `void *ptr;` can point to

- ○ int
- ○ float
- ○ structure
- ○ All of the above

41. Before dereferencing a `void *`, it must be

- ○ Incremented
- ○ Cast to appropriate type
- ○ Freed
- ○ Initialized to NULL

42. Which is illegal? `void *p; *p=10;`

- ○ Legal
- ○ Compile Error
- ○ Warning only
- ○ Runtime Error

43. Which function can accept any data type using `void *`**?**

- ○ malloc()
- ○ memcpy()
- ○ qsort()
- ○ All of the above

44. `void fun(void){}` **This function accepts**

- ○ One argument
- ○ Two arguments
- ○ No arguments
- ○ Variable arguments

45. Which keyword allows variable number of arguments?

- ○ extern
- ○ register
- ○ ...
- ○ typedef

46. Functions like printf() use

- ○ Function pointers
- ○ Variadic arguments
- ○ Recursion
- ○ Structures

47. Stack grows

- ○ Upward on all systems
- ○ Downward on all systems
- ○ Depends on architecture
- ○ Randomly

48. Which statement is true about recursion?

- ○ Every recursive solution has an iterative equivalent.
- ○ Recursion is always faster.
- ○ Recursion never uses stack memory.
- ○ Recursion cannot return values.

49. In embedded systems, interrupt service routines (ISRs) are often registered using

- ○ Arrays
- ○ Function Pointers
- ○ Structures
- ○ Unions

50. Which of the following best describes the execution order of recursive function calls?

- ○ First Called → First Returned (FIFO)
- ○ Last Called → First Returned (LIFO)
- ○ Random Order
- ○ Compiler-dependent